

PostgreSQL

PostgreSQL is a free and open-source database management system used to store, organize and manage data efficiently. It is one of the most popular databases because it is reliable, secure and can handle large amounts of data.

- Supports SQL queries to store, retrieve and manage data.
- Allows transactions to keep data accurate and consistent.
- **Supports JSON and JSONB data for modern applications.**
- Uses Multi-Version Concurrency Control (MVCC) to allow many users to work with the database at the same time.
- Can be extended with custom data types, functions and extensions.

Create a Database

List databases:

```
\l
```

Create a database:

```
CREATE DATABASE company_db;
```

Connect to it:

```
\c company_db
```

Create a Table

```
CREATE TABLE employees (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    department VARCHAR(50),  
    salary NUMERIC(10,2)  
);
```

Explanation:

- SERIAL → auto-increment integer
- PRIMARY KEY → unique identifier
- VARCHAR(100) → text up to 100 characters
- NUMERIC(10,2) → decimal number

Insert Data

```
INSERT INTO employees (name, department, salary)
```

```
VALUES
```

```
('Alice', 'IT', 50000),
```

```
('Bob', 'HR', 45000),
```

```
('Charlie', 'IT', 60000);
```

View data:

```
SELECT * FROM employees;
```

PostgreSQL Data Types:

1. Numeric Data Types

Used to store **numbers**.

Data Type	Description	Example
SMALLINT	Small integer (2 bytes)	100
INTEGER or INT	Standard integer (4 bytes)	25000
BIGINT	Large integer (8 bytes)	9876543210
DECIMAL(p,s)	Exact decimal values	1234.56
NUMERIC(p,s)	Same as DECIMAL	9999.99
REAL	Single precision floating-point	12.45
DOUBLE PRECISION	Double precision floating-point	12345.6789

2. Character Data Types

Used to store **text**.

Data Type	Description
CHAR(n)	Fixed-length string
VARCHAR(n)	Variable-length string (maximum length n)
TEXT	Unlimited-length text

3. Date and Time Data Types

Used to store **dates and times**.

Data Type	Example
DATE	2026-07-04
TIME	15:30:00
TIMESTAMP	2026-07-04 15:30:00
INTERVAL	2 days

INTERVAL Data Type in PostgreSQL

The **INTERVAL** data type is used to store a **duration (time difference)** instead of a specific date or time.

For example:

- 5 days
- 2 hours
- 30 minutes
- 1 year 6 months

Unlike DATE or TIMESTAMP, which store a point in time, INTERVAL stores **how much time has passed or should be added/subtracted**.

Example 2: Add an Interval to a Date

```
SELECT DATE '2026-07-04' + INTERVAL '10 days';
```

Output

2026-07-14 00:00:00

Explanation:

2026-07-04

+ 10 days

2026-07-14

Example 3: Subtract an Interval

```
SELECT DATE '2026-07-04' - INTERVAL '7 days';
```

Output

2026-06-27 00:00:00

Example 4: Add Hours

```
SELECT TIMESTAMP '2026-07-04 10:30:00'  
      + INTERVAL '2 hours';
```

Output

2026-07-04 12:30:00

Example 5: Add Minutes

```
SELECT TIMESTAMP '2026-07-04 10:30:00'  
      + INTERVAL '45 minutes';
```

Output

2026-07-04 11:15:00

4. Boolean Data Type

Stores **TRUE**, **FALSE**, or **NULL**.

Example

```
CREATE TABLE Employee (  
    id INT,  
    is_active BOOLEAN  
);
```

Insert

```
INSERT INTO Employee VALUES  
(1,TRUE),  
(2,FALSE),  
(3,NULL);
```

Output

id is_active

1 TRUE

2 FALSE

3 NULL

5. Binary Data Type (BYTEA)

Stores **binary data** such as:

- Images
- PDF files
- Audio
- Video

- Documents

Example

```
CREATE TABLE Files (  
  id INT,  
  file_data BYTEA  
);
```

BYTEA stores the raw bytes of the file.

6. JSON Data Types

Used to store **JSON documents**.

Type	Description
JSON	Stores JSON exactly as entered
JSONB	Stores JSON in a binary format for faster querying and indexing

6. JSON Data Types

Used to store **JSON documents**.

Type	Description
------	-------------

JSON	Stores JSON exactly as entered
------	--------------------------------

JSONB	Stores JSON in a binary format for faster querying and indexing
-------	---

Example

```
CREATE TABLE Customer (  
  id INT,  
  details JSONB  
);
```

Insert

```
INSERT INTO Customer VALUES  
(
```

```
1,  
{  
  "name": "Alice",  
  "age": 22,  
  "city": "Chennai"  
}
```

Retrieve

```
SELECT details->>'name'  
FROM Customer;
```

Output

Alice

Why use JSONB?

- Faster searches
- Supports indexing
- Removes unnecessary whitespace
- Preferred for most applications

7. UUID Data Type

UUID stands for **Universally Unique Identifier**.

Example UUID

550e8400-e29b-41d4-a716-446655440000

Create table

```
CREATE TABLE Orders (  
  order_id UUID,  
  customer_name VARCHAR(50)  
);
```

Useful for:

- Distributed systems
- APIs

- Secure unique IDs

8. Array Data Type

PostgreSQL allows storing **multiple values in one column**.

Example

```
CREATE TABLE Student (
```

```
id INT,
```

```
marks INT[]
```

```
);
```

Insert

```
INSERT INTO Student VALUES  
(1,{'80,90,95'});
```

Retrieve

```
SELECT *  
FROM Student;
```

Output

id	marks
1	{80,90,95}

Another example

```
CREATE TABLE Employee (
```

```
id INT,
```

```
skills TEXT[]
```

```
);
```

Insert

```
INSERT INTO Employee VALUES  
(  
1,  
'{"Java","Python","SQL"}'  
);
```

Output

{Java,Python,SQL}

PostgreSQL Geometric Data Types:

PostgreSQL provides built-in Geometric Data Types to store and manipulate geometric objects like points, lines, circles, polygons, etc. These are useful in maps, GIS, CAD software, games, and location-based applications.

Data Type Stores		Example
POINT	One point	(10,20)
LINE	Infinite line	{1,-1,0}
LSEG	Line segment	[(1,2),(5,6)]
BOX	Rectangle	((10,10),(0,0))
PATH	Connected points	[(0,0),(5,5),(10,2)]
POLYGON	Closed shape	((0,0),(5,0),(5,5),(0,5))
CIRCLE	Circle	<(5,5),3>

Real-World Example

Suppose you're building a **food delivery application**:

Requirement	PostgreSQL Data Type
-------------	----------------------

Restaurant location	POINT
---------------------	-------

Delivery route	PATH
----------------	------

Delivery area	POLYGON
---------------	---------

Delivery radius (5 km)	CIRCLE
------------------------	--------

City boundary	POLYGON
---------------	---------

Building outline	BOX
------------------	-----

Road segment	LSEG
--------------	------

These geometric data types allow PostgreSQL to efficiently store and query spatial information without needing separate columns for every coordinate.

PostgreSQL CREATE DOMAIN and CREATE TYPE

PostgreSQL allows you to create your own custom data types using:

1. **CREATE DOMAIN** – Creates a custom data type with constraints.
2. **CREATE TYPE** – Creates a new data type (usually a composite type).

1. CREATE DOMAIN

What is a Domain?

A **Domain** is a **custom data type** based on an existing data type with additional rules (constraints).

Instead of writing the same constraints repeatedly, define them once and reuse them.

Real-Life Example

Suppose every student's:

- First name must not be NULL
- First name must not contain spaces

Instead of writing this in every table, create a domain.

Without Domain

```
CREATE TABLE marksheet (  
  
    student_id SERIAL PRIMARY KEY,  
  
    first_name VARCHAR NOT NULL  
    CHECK(first_name !~ '\s'),  
  
    last_name VARCHAR NOT NULL  
    CHECK(last_name !~ '\s'),  
  
    marks INT  
  
);
```

Notice the same validation must be repeated for every table.

Creating a Domain

```
CREATE DOMAIN student_detail  
AS VARCHAR  
NOT NULL  
CHECK (VALUE !~ '\s');
```

Explanation

Keyword	Meaning
student_detail	Domain name
VARCHAR	Base data type
NOT NULL	Value cannot be NULL
VALUE	Refers to the value being inserted

Keyword	Meaning
!~ '\s'	Must not contain whitespace

Using the Domain

CREATE TABLE marksheet (

student_id SERIAL PRIMARY KEY,

first_name student_detail,

last_name student_detail,

marks INT,

email VARCHAR

);

No need to repeat the constraints.

Insert Valid Data

INSERT INTO marksheet

VALUES

(

1,

'Raju',

'Singh',

95,

'raju@gmail.com'

);

Output

INSERT 0 1

Insert Invalid Data

```
INSERT INTO marksheet  
VALUES  
(  
2,  
'Raju Kumar',  
'Singh',  
90,  
'raju@gmail.com'  
);
```

Output

ERROR:
value for domain student_detail
violates check constraint

Because

'Raju Kumar'
contains a space.

Why Use CREATE DOMAIN?

Suppose your database has **20 tables**.

Without a domain, you repeat the same constraint 20 times.

With a domain:

- Create it once
- Reuse everywhere
- Easy to maintain
- Ensures consistent validation

2. CREATE TYPE

What is CREATE TYPE?

CREATE TYPE creates a **new data type**.

The most common form is a **Composite Type**, which groups multiple related fields into a single type.

Think of it like a **class or structure** in programming.

Syntax

```
CREATE TYPE type_name AS (
```

```
column1 datatype,
```

```
column2 datatype,
```

```
...
```

```
);
```

Example

Create an Address type.

```
CREATE TYPE address AS (
```

```
street VARCHAR,
```

```
city VARCHAR,
```

```
state CHAR(2),
```

```
zip_code CHAR(6)
```

```
);
```

Now address behaves like a data type.

Using Composite Type

```
CREATE TABLE employee (
```

```
employee_id SERIAL PRIMARY KEY,
```

```
name VARCHAR,
```

```
home_address address,
```

```
office_address address
```

```
);
```

Notice

home_address

is not a VARCHAR.

It stores

- Street
- City
- State
- ZIP code

as one object.

Insert Data

```
INSERT INTO employee
```

```
VALUES
```

```
(
```

```
1,
```

```
'Alice',
```

```
('12 MG Road','Chennai','TN','600001'),
```

```
('45 Anna Salai','Chennai','TN','600002')  
);
```

View Data

```
SELECT * FROM employee;
```

Output

employee_id	name	home_address	office_address
1	Alice	("12 MG Road",Chennai,TN,600001)	("45 Anna Salai",Chennai,TN,600002)

Access Individual Fields

```
SELECT
```

```
(home_address).city,
```

```
(home_address).zip_code
```

```
FROM employee;
```

Output

city	zip_code
Chennai	600001